

Dev Team — Manual de Usuario

Version del plugin: 2.10.x · **Integrantes del equipo:** 15 · **Comandos:** 22

Dev Team es un **equipo completo de desarrollo de software formado por asistentes de inteligencia artificial**. Funciona como una empresa de software en miniatura: hay alguien que escribe lo que el negocio necesita, alguien que diseña las pantallas, alguien que decide como se construye, programadores, un equipo de pruebas, un encargado de seguridad, uno de publicar versiones y uno que documenta. Tu hablas con ellos en lenguaje normal y ellos hacen el trabajo, pidiendote confirmacion antes de cada paso importante.

No necesitas saber programar para usarlo. Este manual esta escrito para cualquier persona: quien recién empieza, quien dirige un area, quien nunca ha abierto una herramienta tecnica. La parte tecnica (para quien instala y configura) esta al final, separada, en el **Anexo tecnico**.

Indice

Parte 1 — Para todas las personas

1. En dos minutos: como se usa
2. Conoce al equipo
3. Las palabras que vas a escuchar
4. Las reglas de la casa
5. Que puedes pedir (lista de comandos)

Parte 2 — Situaciones reales, paso a paso

- Caso 1: Tengo una idea y quiero empezar un proyecto
- Caso 2: Tengo un documento de requerimientos
- Caso 3: Ya tengo un proyecto andando y quiero que el equipo lo tome
- Caso 4: Necesito una funcionalidad nueva, de principio a fin
- Caso 5: Quiero ver como se vera una pantalla antes de construirla
- Caso 6: Algo no funciona (reportar y corregir un error)
- Caso 7: Algo falla y nadie sabe en que parte esta el problema
- Caso 8: Pruebas — como se comprueba que todo funciona
- Caso 9: Base de datos — revisar, comparar y preparar cambios
- Caso 10: Publicar una version en un ambiente (pase)
- Caso 11: Revisar la seguridad
- Caso 12: La memoria del proyecto (wiki)
- Caso 13: Ver al equipo trabajar
- Caso 14: Trabajar por sprints (Scrum)
- Caso 15: Hacer que un integrante piense mas (o gaste menos)

Parte 3 — Preguntas frecuentes y problemas comunes

- Preguntas frecuentes
- Si algo no anda

Anexo tecnico (para quien instala y configura)

- A. Instalacion y actualizacion
- B. Configuracion del proyecto
- C. La carpeta de coordinacion
- D. Cuidar el consumo
- E. Solucion de problemas tecnicos

Parte 1 — Para todas las personas

1. En dos minutos: como se usa

1. Alguien de tu area instala el plugin una vez (ver Anexo A).
2. Abres Claude Code **en la carpeta de tu proyecto** (o en la carpeta donde quieres crearlo).
3. Escribes esto y describes lo que necesitas, como se lo dirias a una persona:

```
/dev-team:start
```

Ejemplos de lo que puedes escribir despues (o en la misma linea):

Lo que escribes	Lo que pasa
<code>/dev-team:start</code>	El equipo mira donde estas y te propone que hacer
<code>/dev-team:start</code> quiero un sistema para avisar a clientes con pagos por vencer	Arranca un proyecto nuevo desde tu idea
<code>/dev-team:start</code> los analistas necesitan filtrar las cobranzas por fecha	Convierte tu necesidad en trabajo para el equipo
<code>/dev-team:start</code> al pasar a la pagina 2 aparecen registros repetidos	Registra el problema y organiza su correccion
<code>/dev-team:start</code> ¿en que estamos?	Te resume el estado del proyecto

Tres cosas que siempre van a pasar:

- **Te presentan el plan antes de hacer nada** que cambie el producto. Tu dices "adelante", "cambia esto" o "mejor de otra forma".
- **Todo queda con pruebas y con fotos** (capturas de pantalla) que demuestran que funciona.
- **Nada se publica sin pasar controles.** El equipo se controla a si mismo.

Si solo tienes una pregunta rapida ("¿que estados tiene una solicitud?"), preguntala directo, sin /dev-team:start . Es mas rapido y consume menos.

2. Conoce al equipo

Piensa en una empresa de software pequeña. Estos son sus 15 integrantes, con lo que cada uno hace y cuando te vas a cruzar con ellos.

Integrante	Que hace, en palabras simples	Cuando aparece
 setup (el que prepara la oficina)	Revisa que la computadora tenga todo lo necesario e instala lo que falte, con tu permiso	Al empezar un proyecto o cuando algo no esta instalado
 product-owner (el dueño del producto)	Escribe lo que el negocio necesita en lenguaje claro: historias, errores, prioridades. Lleva el tablero de tareas	Siempre que pides algo nuevo o reportas un problema
 architect (el arquitecto)	Decide como se construye: en cuantas partes, con que tecnologias, como se conectan	Al crear un proyecto o ante una decision estructural
 ui-designer (el diseñador)	Propone como se veran las pantallas (colores, tipografia, distribucion) y te deja elegir	Antes de construir una pantalla nueva
 lead (el jefe de equipo)	Coordina, arma el plan, reparte el trabajo, exige que se cumplan los controles y es el unico que integra los cambios al producto	Detras de cada tarea; lo ves al aprobar planes
 backend (programador de "la cocina")	Construye la logica y los servicios que no se ven	Al construir funcionalidades
 frontend (programador de pantallas)	Construye lo que el usuario ve y toca	Al construir pantallas
 dba (el de las bases de datos)	Cuida los datos: estructura, cambios, comparaciones, preparacion de cambios para publicar	Cambios de datos y publicaciones
 qa (jefe de pruebas)	Planifica las pruebas, reparte a sus dos especialistas y da el veredicto final: aprobado o rechazado	Al terminar cualquier trabajo
 qa-frontend (probador de pantallas)	Prueba las pantallas como un usuario real, con fotos de cada paso	Pruebas de pantallas
 qa-backend (probador de servicios)	Prueba que los servicios respondan exactamente lo prometido	Pruebas de servicios y datos
 release-manager (encargado de publicaciones)	Arma la solicitud formal para publicar una version en un ambiente, revisa que los cambios de datos esten bien hechos y puede rechazarlos	Cada vez que hay que "pasar" algo a un ambiente

Integrante	Que hace, en palabras simples	Cuando aparece
 infra (el de la infraestructura)	Prepara los servidores, los contenedores y la automatizacion para publicar	Publicaciones y configuracion de ambientes
 cybersec (seguridad)	Audita la seguridad y reporta hallazgos. Nunca toca el codigo: lo corrige el responsable	Cambios sensibles: claves, datos personales, accesos
 tech-writer (el documentador)	Mantiene la documentacion y la memoria del proyecto (la wiki)	Al cerrar funcionalidades y al final del dia

Sobre el costo: cada integrante usa un "cerebro" (modelo de IA) del tamaño justo para su trabajo, para no gastar de mas. El que prepara la oficina y el documentador usan el mas economico; el resto uno intermedio; el arquitecto puede subirse a uno mas potente si el proyecto lo amerita (ver [Caso 15](#)).

3. Las palabras que vas a escuchar

El equipo intenta hablar claro, pero hay palabras del oficio que conviene conocer:

Palabra	Que significa
Historia de usuario (HU)	Una necesidad escrita desde el punto de vista de quien la usa: "Como analista quiero filtrar por fecha para encontrar los pagos de un periodo". Lleva criterios que dicen cuando esta lista
Criterio de aceptacion	Cada condicion concreta que debe cumplirse para dar la historia por terminada. Se escriben como "Dado..., cuando..., entonces..."
Tablero de tareas (tracker)	Donde viven las historias y los errores: GitHub o Azure DevOps, segun tu empresa
Sprint	Un periodo corto de trabajo (normalmente 2 semanas) con un objetivo claro
Bug	Un error: algo que deberia funcionar de una forma y funciona de otra
Ambiente	Una copia del sistema para un proposito: desarrollo, pruebas, demostracion, produccion (la que usan los clientes)
Pase	Publicar una version en un ambiente. Lleva una solicitud formal
Evidencia	Fotos (capturas de pantalla), videos cortos y registros que demuestran lo que se probó y que paso
Informe de conformidad	El aviso de quien publico una version: que quedo instalado, en que ambiente, y que esta funcionando. Sin este aviso, pruebas no arranca
Repositorio	La carpeta donde vive el codigo de un componente, con su historial de cambios
Rama	Una copia de trabajo separada dentro del repositorio, para que cada cambio no interfiera con los demas

Palabra	Que significa
Integrar (merge)	Llevar un cambio terminado y aprobado al producto principal. Solo lo hace el jefe de equipo
Plan primero	La regla de que nadie cambia nada sin mostrarte antes el plan y esperar tu "adelante"
Nota de traspaso (handoff)	La forma en que los integrantes se pasan trabajo entre ellos: un mensaje escrito que queda guardado. Todo es trazable
Wiki	La memoria del proyecto: paginas cortas con lo que se decidio y por que

4. Las reglas de la casa

Estas reglas estan siempre activas. No hay que pedir las.

1. **Plan primero.** Antes de construir o corregir algo, el jefe de equipo te presenta el plan (que, quien, donde, riesgos) y espera tu confirmacion. Nada se ejecuta sin tu OK.
2. **Todo se escribe en lenguaje de negocio.** Las historias y los errores los entiende cualquier persona. Lo tecnico va aparte, entre los integrantes.
3. **Se trabaja por sprints**, con un objetivo por sprint, tareas estimadas y prioridades por valor para el negocio.
4. **Nada se integra al producto sin la aprobacion de pruebas**, con evidencia.
5. **Regla de oro (no se cambia nunca): pruebas no valida sin el informe de conformidad.** Si nadie confirmo que quedo instalado y funcionando, no se prueba. En la computadora del desarrollador, solo si todo el sistema esta levantado completo. Nadie puede saltarse esto, ni siquiera el jefe de equipo.
6. **A la primera falla, se reporta.** Si algo esta caido o no funciona al primer intento, pruebas toma la foto, marca el bloqueo y avisa. No insiste, no busca atajos, no toca nada.
7. **Pruebas no arregla errores.** Reproduce, documenta y reporta. Corregir es trabajo del programador responsable. Asi la evidencia es objetiva.
8. **Toda prueba deja evidencia, y la evidencia se ve dentro del ticket.** Las fotos y videos aparecen dentro del error o la historia, no como un enlace aparte. Se guardan en la carpeta del proyecto y, en GitHub, en un espacio apartado que existe solo para eso. Nunca se mezclan con el codigo.
9. **Seguridad revisa lo sensible** (accesos, claves, datos personales) y nunca modifica codigo: reporta y el responsable corrige.
10. **Solo el jefe de equipo integra cambios al producto.** Cada integrante trabaja en su propia copia, en una tarea a la vez.
11. **Publicar una version pasa por un control.** El encargado de publicaciones revisa el paquete completo y puede rechazar cambios de datos mal preparados.
12. **Los programadores preguntan antes de pedir revision formal** de un cambio. No todo lo lleva.
13. **Solo el jefe de equipo puede pedir ayuda a otros integrantes.** Los demas hacen su trabajo directo. Esto evita que el costo se dispare.
14. **Nada de nombres ni datos escritos a mano.** Personas, correos, responsables y rutas salen de la configuracion del proyecto o se te preguntan.

15. **Todo queda registrado automaticamente.** La actividad del equipo se guarda sola y alimenta la oficina virtual y las metricas.

5. Que puedes pedir (lista de comandos)

Recuerda: con `/dev-team:start` y una frase basta. Esta lista es para cuando ya sabes exactamente que quieres.

Todos los días

Escribes	Que consigues
<code>/dev-team:start</code>	El punto de partida: mira tu situacion y te guia
<code>/dev-team:status</code>	Resumen del proyecto: sprint, pendientes, bloqueos
<code>/dev-team:sync</code>	Sincroniza con el tablero de tareas (trae lo nuevo, sube avances)
<code>/dev-team:inbox</code>	Un integrante revisa las notas de traspaso que tiene pendientes

Proyectos

Escribes	Que consigues
<code>/dev-team:new-project {idea o documento}</code>	Proyecto nuevo completo: arquitectura, carpetas, tareas, memoria
<code>/dev-team:onboard {nombre}</code>	El equipo toma un proyecto que ya existe
<code>/dev-team:setup</code>	Revisa e instala lo que la computadora necesita

Trabajo

Escribes	Que consigues
<code>/dev-team:refine {pedido}</code>	El dueño del producto convierte tu pedido en historias en el tablero
<code>/dev-team:assign-task</code>	El jefe de equipo reparte el trabajo (con plan primero)
<code>/dev-team:handoff</code>	Crear una nota de traspaso entre integrantes

Calidad

Escribes	Que consigues
<code>/dev-team:test-plan {HU}</code>	El plan de pruebas de una historia
<code>/dev-team:e2e {HU}</code>	Pruebas de una historia con fotos por criterio (ver Caso 8)
<code>/dev-team:e2e run</code>	Comprobar que todo lo que funcionaba sigue funcionando
<code>/dev-team:review-pr {n}</code>	Revisar un cambio propuesto

Escribes	Que consigues
<code>/dev-team:security-audit</code>	Auditoria de seguridad
<code>/dev-team:git-check</code>	Revisar el estado del repositorio antes de guardar cambios

Publicaciones y operacion

Escribes	Que consigues
<code>/dev-team:db-health</code>	Chequeo de salud de la base de datos
<code>/dev-team:deploy-check</code>	¿Esta listo para publicarse?
<code>/dev-team:pase {ambiente}</code>	La solicitud de pase completa (documento + cambios de datos revisados)
<code>/dev-team:document {tema}</code>	Actualizar documentacion

Memoria y visibilidad

Escribes	Que consigues
<code>/dev-team:wiki query {pregunta}</code>	Preguntarle a la memoria del proyecto
<code>/dev-team:team-office</code>	Ver al equipo trabajar en vivo (oficina virtual)
<code>/dev-team:team-metrics</code>	Quien hizo que, cuanto tardo, cuanto consumo

Parte 2 — Situaciones reales, paso a paso

Cada caso muestra la situacion, **que escribes exactamente**, que hace el equipo y que recibes al final. Los ejemplos son de una empresa financiera ficticia, pero aplican a cualquier negocio.

Caso 1 — Tengo una idea y quiero empezar un proyecto

Situacion: quieres un sistema que avise a los clientes cuando tienen pagos por vencer. Tienes la idea, no tienes nada mas.

Que escribes:

```
/dev-team:new-project sistema de avisos de cobranza: avisa por correo a los
clientes con pagos proximos a vencer, con plantillas configurables y reportes
de envio. Lo usan los analistas de cobranza. Se conecta con nuestro sistema
central.
```

Que hace el equipo:

1. **setup** revisa que la computadora tenga lo necesario y te pide UNA confirmacion para instalar lo que falte.
2. **El arquitecto** analiza la idea y te presenta una propuesta concreta, en una pagina: en cuantas partes se divide el sistema, que hace cada una, como se guardan los datos y en que orden se construye. Por ejemplo:

Propuesta – Avisos de Cobranza

Partes: 1) Avisos y plantillas 2) Programador de vencimientos

3) Puerta de entrada 4) Pantallas de gestion

Datos: cada parte con su propia base de datos

Etapas: 1) plantillas + envio manual 2) envio automatico 3) reportes

3. **Tu decides.** Puedes decir "prefiero todo en un solo lugar", "quita la puerta de entrada", "usa la base de datos que ya tenemos". El arquitecto ajusta y vuelve a presentar hasta tu OK.
4. Eliges donde vivira el tablero de tareas (GitHub o Azure DevOps) y el equipo crea las carpetas de cada parte con todo lo basico ya armado.
5. **El dueño del producto** convierte la idea en historias reales en tu tablero, escritas para que cualquiera las entienda:

"Como analista de cobranza quiero configurar la plantilla del aviso de vencimiento para adaptar el tono a cada cliente" — con sus criterios de aceptacion, estimacion y prioridad.

6. Se crea la memoria del proyecto (wiki) y se activan las metricas.

Recibes: un proyecto listo para trabajar, un tablero con las historias priorizadas y la arquitectura documentada. El siguiente paso tipico es `/dev-team:assign-task` para empezar la primera historia.

Caso 2 — Tengo un documento de requerimientos

Situacion: el cliente entrego un documento (Word, PDF o texto) con lo que necesita.

Que escribes:

```
/dev-team:new-project docs/Requerimientos_Portal_Proveedores.docx
```

Que cambia respecto al Caso 1:

- El arquitecto **lee el documento completo** y relaciona cada requerimiento con una parte del sistema y una etapa. Antes de proponer, **te señala vacios y ambigüedades**: "el documento no dice si los proveedores entran con su clave de la empresa o con una clave propia. ¿Cual es?".
- El dueño del producto arma las historias **indicando de que requerimiento sale cada una** (por ejemplo, "RQ-07 → Consultar estado de facturas"), para que nada del documento quede afuera.
- Si el documento trae bocetos de pantallas, el diseñador los toma como referencia (ver [Caso 5](#)).

Mientras mas decisiones traigas tomadas (tablero, tecnologias preferidas), menos preguntas te hara el equipo. Todo lo que no traigas, el arquitecto lo propone y tu solo apruebas o ajustas.

Caso 3 — Ya tengo un proyecto andando y quiero que el equipo lo tome

Situacion: un sistema real que ya existe, con varias partes, tareas abiertas y una base de datos en uso. Quieres que el equipo lo opere desde hoy.

Que escribes (parado en la carpeta que contiene el proyecto):

```
/dev-team:onboard backoffice
```

Que hace el equipo:

1. **setup** revisa la computadora y te pregunta si el tablero es GitHub, Azure DevOps o si por ahora se trabaja solo en local.
2. Reconoce las partes del sistema, con que estan hechas y como estan organizadas. **Nunca propone cambiar tu forma de organizarlo.**
3. Configura el acceso del encargado de bases de datos (te pide los datos de conexion una vez; se guardan protegidos, fuera del codigo) y prueba que conecta.
4. Trae las tareas reales del tablero y arma el sprint actual.
5. Arma un mapa de donde esta cada cosa, para que nadie pierda tiempo buscando.
6. Crea la memoria del proyecto y le carga lo que descubrio.
7. Si el proyecto no tiene un espacio de pruebas automaticas, te ofrece crearlo.

Recibes: un equipo que conoce tu proyecto y puede trabajar. Prueba con `/dev-team:status` para ver el resumen.

Si el proyecto tiene varias partes en carpetas separadas, abre Claude Code siempre desde la carpeta que las contiene a todas. Si trabajas "solo local", las tareas viven en un archivo del proyecto y puedes conectar un tablero despues con `/dev-team:setup tracker`.

Caso 4 — Necesito una funcionalidad nueva, de principio a fin

Situacion: los analistas necesitan filtrar las cobranzas por rango de fechas.

Que escribes:

/dev-team:refine los analistas necesitan filtrar las cobranzas por rango de fechas

Que hace el equipo:

0. Plan primero. El jefe de equipo te presenta el plan y espera tu OK:

Plan propuesto – Filtro de fechas en cobranzas

Que: filtro desde/hasta en el listado de cobranzas (pantalla y servicio)

Quien: dueño del producto escribe la historia → programador de servicios y
programador de pantallas trabajan en paralelo → pruebas valida → se integra

Riesgos: la consulta por rango puede ser lenta con muchos registros; el encargado
de datos revisara si hace falta un indice

¿Apruebas, ajustas, o lo abordamos de otra forma?

1. El dueño del producto escribe la historia en tu tablero:

Filtrar las cobranzas por rango de fechas

Como analista de cobranzas

quiero filtrar el listado por fecha desde/hasta
para encontrar rapidamente los pagos de un periodo.

Criterios de aceptacion

1. Dado un rango valido, cuando filtro, entonces veo solo cobranzas del rango.
2. Dado un rango invalido (desde mayor que hasta), cuando filtro, entonces veo el mensaje "El rango de fechas no es valido" y el listado no cambia.
3. Dado el filtro activo, cuando lo limpio, entonces vuelvo al listado completo.

Estimacion: 3 puntos

2. **El jefe de equipo reparte:** los dos programadores trabajan a la vez, cada uno en su copia, y el jefe de pruebas prepara el plan de pruebas al mismo tiempo.
3. **Los programadores terminan** y avisan a pruebas **con el informe de conformidad** (que quedo disponible, en que version, y que esta funcionando). Sin ese aviso, pruebas no arranca.
4. **El equipo de pruebas** valida cada criterio: el probador de pantallas recorre la aplicacion como un usuario y deja fotos de cada paso; el probador de servicios comprueba las respuestas. El jefe de pruebas consolida un solo veredicto: **APROBADA** o **RECHAZADA**.
5. **El jefe de equipo** verifica que todo este en orden (pruebas aprobo, seguridad si aplica) e integra el cambio al producto.
6. La historia pasa a "terminada" en tu tablero, con la evidencia adentro.
7. **El documentador** deja registro en la memoria del proyecto.

Recibes: la funcionalidad integrada, la historia cerrada con pruebas y fotos, y la memoria del proyecto al dia.

Caso 5 — Quiero ver como se vera una pantalla antes de construirla

Situacion: necesitas una pantalla nueva de "Resumen de cobranzas" y quieres elegir el diseño antes de que alguien la programe.

Que escribes:

```
/dev-team:start necesito diseñar la pantalla de resumen de cobranzas antes de construirla
```

Que hace el diseñador:

1. Mira como se ven hoy tus pantallas (colores, tipografia, estilo) para partir de tu realidad, no de cero.
2. Te entrega **dos o tres propuestas** como paginas que abres en tu navegador y se ven como la pantalla real:

propuesta-A-densa.html	"para leer muchos datos: tabla e indicadores arriba"
propuesta-B-aireada.html	"para leer rapido: tarjetas y un grafico"
design-spec.md	colores (con contraste verificado), letras, espacios, y como se ve cuando carga, cuando falla, cuando esta vacia

3. Eliges ("la B, pero con los indicadores de la A"). El diseñador arma la version final y se la pasa al programador de pantallas indicando que es fijo y que puede adaptarse.
4. El programador construye **exactamente eso**.

Recibes: pantallas decididas por ti, con algo visible antes de gastar en construccion. Cero "no era asi como lo imaginaba".

Caso 6 — Algo no funciona (reportar y corregir un error)

Situacion: "al pasar a la pagina 2 del listado de cobranzas aparecen registros que ya vi en la pagina 1".

Que escribes:

```
/dev-team:start al pasar a la pagina 2 del listado de cobranzas aparecen registros que ya vi en la pagina 1
```

Que hace el equipo (registrar → corregir → volver a probar → cerrar):

1. **El jefe de equipo** hace una primera evaluacion: que parte parece afectada y que tan grave es. No corrige nada todavia.
2. **Pruebas reproduce el problema** como un usuario, con fotos numeradas de cada paso (00-listado-pagina-1.png , 01-pagina-2-repetidos.png). Si no logra reproducirlo al primer intento o algo esta caido, toma la foto, marca el bloqueo y avisa. No insiste.
3. **El dueño del producto registra el error en el tablero**, en lenguaje claro:

"El listado de cobranzas muestra pagos repetidos al cambiar de pagina" Pasos como usuario, que se esperaba y que paso, gravedad e impacto, y las **fotos dentro del ticket**. Esto ocurre **antes** de hablar de quien lo corrige, aunque ya se sospeche la causa.

4. **Plan primero:** el jefe de equipo te presenta el plan de correccion y espera tu OK.
5. **El programador** corrige en su copia; **pruebas escribe una prueba automatica** que reproduce el error, para que no vuelva a pasar sin que nadie se entere.
6. **Pruebas vuelve a probar** el mismo recorrido con la correccion instalada, con una tanda de fotos nueva, y da el veredicto.
7. **El dueño del producto comenta en el mismo ticket** (que se corrigio, el veredicto, las fotos nuevas) y **te pregunta si lo cierra**. Nunca lo cierra solo.

Recibes: el error corregido, con toda la historia y las fotos dentro del ticket, y una prueba automatica que impide que vuelva.

Caso 7 — Algo falla y nadie sabe en que parte esta el problema

Situacion: "el ingreso funciona si se prueba directo contra el servicio, pero desde la aplicacion instalada falla". Puede ser la pantalla, la puerta de entrada, el servicio de accesos o la configuracion de los servidores. Nadie sabe donde.

Que escribes:

```
/dev-team:start el ingreso falla desde la aplicacion instalada (error generico),  
pero el mismo usuario funciona probando directo contra el servicio
```

Que hace el equipo:

1. **El jefe de equipo** te presenta el plan de investigacion:

```
Plan – El ingreso falla solo desde la aplicacion  
1. Pruebas reproduce el problema en el ambiente de pruebas, con fotos y el  
registro de lo que la aplicacion intento hacer  
2. Se aísla por capas, varios integrantes a la vez, cada uno en su parte:  
– probador de servicios: ¿responde bien directo? ¿y pasando por la puerta de  
entrada?  
– infraestructura: ¿la configuracion instalada es la correcta?  
3. Con la parte culpable identificada → se registra el error → correccion dirigida  
Nada se toca hasta tu OK.
```

2. **Pruebas reproduce UNA vez** con evidencia: la foto del error y el registro que muestra que respondio realmente el sistema.
3. **Investigacion en paralelo**, cada uno en su parte. Ejemplo real: directo al servicio responde bien; pasando por la puerta de entrada responde "metodo no permitido". Infraestructura revisa la configuracion instalada (no la de desarrollo) y encuentra que no dirige las llamadas al lugar correcto.

4. **El dueño del producto registra el error** apuntando a la parte real, en lenguaje claro, con toda la evidencia.
5. Corrige el responsable de esa parte y **pruebas vuelve a probar el recorrido completo sobre el sistema realmente instalado**, no sobre la version de escritorio del programador.

Por que funciona: comparar "directo" contra "pasando por el medio" aísla la capa sin leer una linea de codigo; y probar siempre sobre lo realmente instalado evita que este tipo de errores se esconda durante días.

Caso 8 — Pruebas: como se comprueba que todo funciona

El equipo de pruebas trabaja con una meta clara: **equivocarse como maximo una vez cada mil criterios revisados**. Para eso no "mira" la pantalla y opina: cada criterio se comprueba con una verificacion concreta, queda una foto con el elemento resaltado, y la historia solo se aprueba si pasa 7 controles. Tu no tienes que instalar ni configurar nada: las herramientas de prueba vienen incluidas.

Pedir el plan de pruebas de una historia:

```
/dev-team:test-plan HU-42
```

→ una tabla con cada criterio: cual se probara de forma automatica, cual a mano, que casos extremos se agregan y que datos de prueba hacen falta.

Validar una historia (lo mas habitual):

```
/dev-team:e2e HU-42
```

→ Primero pruebas confirma que lo que va a probar es lo que se dijo que se instalo (el informe de conformidad). Luego recorre la aplicacion como lo haria un usuario y, por cada criterio, deja **tres fotos** (antes, la accion, el resultado con el elemento resaltado) y una **comprobacion** que dice si se cumple o no. Con eso arma un **informe por criterio**, como este:

```
CA-2 · Mostrar error con rango invalido – CUMPLE
| Paso | Accion                                     | Foto                                |
| 1    | Estado inicial del filtro                 | ca2-01-inicial.png                 |
| 2    | Ingreso rango 31/02 → 01/01 y presiono Filtrar | ca2-02-accion.png                 |
| 3    | Mensaje "El rango de fechas no es valido" visible | ca2-03-resultado.png              |
Comprobacion: el texto esta visible → OK · Sin errores ocultos · Sin llamadas fallidas
```

Despues escribe las pruebas automaticas, una por criterio y con el mismo nombre, para que ese criterio quede protegido de aqui en adelante.

Los 7 controles para aprobar. El jefe de pruebas revisa, en este orden, que:

1. Todos los criterios tienen su comprobacion y su foto.

2. Las pruebas automaticas de la historia pasaron **dos veces seguidas**. Si una pasa una vez y falla otra, no cuenta: se aparta y se investiga.
3. Lo que ya funcionaba sigue funcionando.
4. Las pantallas clave se ven igual que la version aprobada; si cambiaron, el diseñador o el dueño del producto lo aprobo.
5. Las pantallas nuevas son usables por personas con discapacidad (sin fallas graves de accesibilidad).
6. Los servicios responden exactamente lo que prometen: se les envian cientos de solicitudes validas e invalidas generadas automaticamente y ninguna los rompe.
7. Durante el recorrido no hubo errores ocultos ni llamadas fallidas.

Si falla cualquiera, la historia vuelve **RECHAZADA** con el informe. No existe "aprobada con observaciones": una observacion es un error (se registra) o no es nada.

Comprobar que todo sigue funcionando (por ejemplo antes de publicar):

```
/dev-team:e2e run
```

→ corre todas las pruebas dos veces. Cada fallo se clasifica: o es un **error del producto** (se registra con evidencia) o es una **prueba fragil** (se aparta y se repara la prueba, nunca el producto).

Otras cosas que puedes pedirle a pruebas:

```
/dev-team:e2e plan HU-42          # explora la aplicacion y propone el plan de pruebas
/dev-team:e2e generate HU-42      # convierte ese plan en pruebas automaticas
/dev-team:e2e heal                # repara pruebas fragiles (nunca cambia lo que debe
                                # cumplirse)
/dev-team:e2e visual              # compara las pantallas clave con la version aprobada
/dev-team:e2e a11y                # revisa la accesibilidad de las pantallas
/dev-team:e2e api HU-42           # somete los servicios de la historia a su contrato
/dev-team:e2e explorar http://localhost:4200 # recorre libremente y reporta lo que
                                # encuentre
```

Ver que tan preciso esta siendo el equipo de pruebas:

```
/dev-team:team-metrics
```

→ entre otras cosas, cuantas historias aprobadas volvieron despues como error. La meta es una de cada mil o menos.

Donde quedan las fotos. Siempre en la carpeta del proyecto, una subcarpeta por historia o error. Cuando hay un error, las fotos se muestran dentro del ticket. Si el proyecto esta en GitHub, se publican en un espacio apartado que existe solo para eso y nunca se mezclan con el codigo.

Caso 9 — Base de datos: revisar, comparar y preparar cambios

Chequeo de salud:

```
/dev-team:db-health full
```

→ estructura, índices que no se usan, consultas lentas, tamaños. Si no hay conexión configurada, la pide una vez y la guarda protegida.

Comparar dos bases de datos (por ejemplo, la de pruebas contra la de producción del cliente, antes de publicar):

```
/dev-team:start compara la base de datos de pruebas contra la de producción del cliente ACME, te paso los accesos de ambas
```

→ el encargado de datos compara **solo leyendo**: jamás escribe en ninguna de las dos. Entrega un informe de diferencias y los archivos para nivelarlas, **generados pero no ejecutados**: tú decides cuándo y dónde correrlos.

Preparar los cambios de datos para publicar. El encargado de datos entrega siempre en el mismo formato, en archivos numerados por tipo de cambio (crear tablas, modificar tablas, vistas, datos, procedimientos, actualizaciones) y organizados por motor y por base de datos. Cada archivo se puede ejecutar varias veces sin romper nada, respeta acentos y ñes, y no depende de nombres de una instalación particular. El encargado de publicaciones lo revisa y puede rechazarlo (ver Caso 10).

Caso 10 — Publicar una versión en un ambiente (pase)

Situación: hay que publicar la versión 2.4.0 de Avisos de Cobranza en el ambiente **Preprod de Peru**, con cambios de datos.

Que escribes:

```
/dev-team:pase preprod PE – Avisos de Cobranza v2.4.0, lleva cambios de base de datos
```

Que hace el encargado de publicaciones:

1. Reune lo que falte: que componentes van y en qué versión exacta (la real, no inventada), y quien aprueba (sale de la configuración, nunca escrito a mano).
2. **Control "pase completo"**: rechaza si faltan componentes, hay dos versiones para lo mismo, hay más de una rama por componente, hay ramas de otro país, o no está claro el tema. El pase se pide una sola vez, completo. Nada de "te mando el resto después".
3. **Revisa los cambios de datos** que entrego el encargado de datos: que estén en el formato acordado, ordenados por motor, base y tipo, y que se puedan ejecutar más de una vez sin romper nada. **Si algo está mal, los rechaza** e indica archivo, línea y regla.

4. Arma el paquete de cambios de datos (`Scripts.zip`) con esa misma organizacion.
5. Llena la **solicitud de pase** en formato simple, de una o dos paginas: para que es, que componentes van y en que version, que temas incluye, que acciones hay que hacer, que ramas se usaron (una por componente, limpias) y que cambios de datos lleva. Sin relatos largos ni detalles tecnicos innecesarios.
6. Deja el **correo listo para enviar** (asunto, tabla de componentes, acciones y la solicitud de aprobacion a la persona correcta). Si tu Outlook esta conectado, lo deja como borrador; si no, como archivo de texto. **Tu lo envias.**
7. Entrega la **carpeta del pase completa**:

```
Release v2.4.0 16julio2026 – Avisos Cobranza/  
├─ Solicitud de Pase Ambientes – Preprod PE.pdf  
├─ Solicitud de Pase Ambientes – Preprod PE.docx  
├─ Scripts.zip  
├─ S3.zip (si aplica)  
└─ correo-pase.txt
```

Cuando lleva documento: certificacion, puente, demostracion (Chile, Peru, Colombia), preprod (Colombia, Peru) y produccion de clientes. Ambientes internos: solo si lo pides.

Despues de publicar: quien instalo emite el **informe de conformidad**. Sin el, pruebas no valida (regla de oro).

Caso 11 — Revisar la seguridad

Que escribes (en el componente que quieres revisar, o indicando cual):

```
/dev-team:security-audit
```

Que revisa seguridad: los riesgos mas comunes de la industria, claves o secretos escritos en el codigo, componentes externos con vulnerabilidades conocidas, y una lista de controles de acceso aprendidos de incidentes reales (por ejemplo: que no haya puertas sin cerrar, que los intentos de ingreso fallidos se limiten de verdad, que los pasos obligatorios no se puedan saltar).

Recibes: un informe con hallazgos clasificados por gravedad (critico, alto, medio, bajo), cada uno con su evidencia y su recomendacion. **Seguridad nunca cambia codigo:** corrige el responsable de la parte afectada y seguridad vuelve a revisar.

Caso 12 — La memoria del proyecto (wiki)

En vez de que el equipo relea decenas de notas viejas cada dia, mantiene una memoria ordenada: una pagina por servicio, por historia, por error, por decision y por publicacion. Los integrantes la leen antes de trabajar. Tu tambien puedes usarla.

```
/dev-team:wiki query ¿por que elegimos guardar los avisos en una base separada?
```

→ responde solo con lo que esta en la memoria, citando las paginas. Si no lo sabe, lo dice.

```
/dev-team:wiki ingest    # al cierre del dia: guarda lo aprendido hoy  
/dev-team:wiki lint      # revisa la salud de la memoria (enlaces rotos, paginas  
viejas)
```

Solo el documentador escribe en la memoria; todos los demas la leen. Si tienes la aplicacion gratuita **Obsidian**, puedes abrir la carpeta de la wiki y ver un mapa visual de como se conecta todo (historias con servicios, errores con correcciones, decisiones con su razon).

Caso 13 — Ver al equipo trabajar

La oficina virtual, en vivo:

```
/dev-team:team-office
```

→ se abre una pagina en tu navegador con los 15 integrantes en sus salas. Cuando uno trabaja, se ve un anillo verde y "escribiendo..." con su tarea debajo; si esta bloqueado, se pone rojo; cuando uno le pasa trabajo a otro, vuela un sobre; cuando termina, confetti. A un costado, el hilo de actividad. Es solo para mirar y no consume nada.

Las metricas, para tomar decisiones:

```
/dev-team:team-metrics
```

→ por integrante: tareas terminadas, cuanto tardaron, cuanto consumieron y en que "cerebro" corren, con alertas concretas ("seguridad consume el 18 % con el 4 % de las tareas: considera bajarle el modelo"). Incluye la precision del equipo de pruebas.

El estado en texto:

```
/dev-team:status
```

Caso 14 — Trabajar por sprints (Scrum)

1. `/dev-team:refine` {lo que se necesita este sprint}
→ el dueño del producto crea o refina las historias, con estimacion
2. Planificacion: el dueño del producto propone el objetivo del sprint y las historias candidatas; el jefe de equipo valida cuanto cabe
3. `/dev-team:assign-task`
→ plan primero y reparto del trabajo
4. Durante el sprint:
`/dev-team:status` cada mañana (tu reunion diaria)
`/dev-team:sync` para reflejar los avances en el tablero
`/dev-team:team-office` si quieres verlo en vivo
(nada entra al sprint en curso sin tu decision explicita)
5. Revision: el dueño del producto verifica historia por historia contra sus criterios; lo que no se termino vuelve al listado, no se arrastra en silencio
6. Retrospectiva: acuerdos de mejora para el siguiente sprint
7. `/dev-team:wiki ingest`
→ el sprint queda en la memoria del proyecto

Caso 15 — Hacer que un integrante piense mas (o gaste menos)

Cada integrante usa un "cerebro" de un tamaño: **economico**, **intermedio** o **potente**. Mas potente piensa mejor pero cuesta mas. Por defecto el equipo ya esta en el minimo razonable. Lo unico que suele valer la pena subir es el arquitecto en proyectos complejos.

Para este proyecto (en el archivo de configuracion del proyecto):

```
"team": { "models": { "architect": "opus" } }
```

Para todos tus proyectos (en tu configuracion personal, `~/ .claude/dev-team.config.json`):

```
{ "team": { "models": { "architect": "opus" } } }
```

Si hay configuracion en el proyecto y en la personal, gana la del proyecto. Los valores son `haiku` (economico), `sonnet` (intermedio) y `opus` (potente). Dos integrantes no se pueden cambiar: el que prepara la oficina y el documentador, que siempre usan el economico.

Parte 3 — Preguntas frecuentes y problemas comunes

Preguntas frecuentes

¿Tengo que saber que integrante hace cada cosa? No. `/dev-team:start` y describe lo que necesitas. El equipo se organiza.

¿Con quien estoy hablando cuando escribo? Siempre responde "la sesion". Es por diseño: tu conversacion principal actua como el coordinador operativo del equipo (arma planes, reparte, exige controles) sin pagar el costo de un integrante aparte. El jefe de equipo como integrante existe para gestion profunda: sprint, prioridades, revision de cambios e integracion al producto. Si quieres invocarlo explicitamente: `/dev-team:assign-task` o escribe "que el lead revise el sprint".

¿Puedo confiar en que no haran nada sin avisarme? Si. Plan primero es regla: todo lo que construye o corrige te presenta el plan y espera tu OK. Solo lo que es de lectura (mirar, resumir, buscar) corre directo.

¿Funciona sin GitHub ni Azure DevOps? Si. En "solo local" las tareas viven en un archivo del proyecto. Puedes conectar un tablero despues con `/dev-team:setup tracker`.

¿Por que pruebas no arregla los errores que encuentra? Por diseño: pruebas reproduce y reporta con evidencia; el programador corrige. Asi la evidencia es objetiva y corrige quien conoce el codigo.

¿Que pasa si pruebas se topa con algo caido? Toma la foto de ese primer intento, marca el bloqueo y avisa de inmediato. No reintenta, no busca atajos, no toca nada. Es ley.

¿Donde quedan las fotos de las pruebas? En la carpeta del proyecto, una subcarpeta por historia o error, con un informe por criterio. Cuando hay un error, se muestran dentro del ticket. En GitHub se publican en un espacio apartado que existe solo para eso y nunca se mezclan con el codigo; un cambio de codigo que traiga fotos adentro se rechaza. En Azure DevOps se adjuntan al ticket y se ven embebidas.

Pruebas dice que no puede abrir el navegador. El navegador de pruebas viene incluido con el plugin desde la version 2.8.0. Casi siempre basta con cerrar y abrir la sesion. Si antes alguien lo configuro a mano, pide `/dev-team:setup playwright` : detecta si hay uno duplicado y dice como quitarlo.

¿El encargado de datos puede escribir en las bases cuando compara dos? No. Solo lee. Los archivos para nivelar se generan y tu decides cuando ejecutarlos.

¿Siempre se pide revision formal de un cambio al terminar? No. Los programadores preguntan primero. Hay entregables que no lo llevan.

¿Necesito Obsidian para la memoria del proyecto? No. Son paginas de texto y el equipo las usa igual. Obsidian solo agrega el mapa visual para las personas.

¿Puedo usar solo una parte del equipo? Si. Cada comando funciona por separado: solo el chequeo de base de datos, solo refinar historias, solo un pase, solo pruebas.

¿El equipo respeta mi proyecto tal como esta? Si. Al tomar un proyecto existente nunca propone cambiar como esta organizado ni con que esta hecho, salvo que lo pidas.

Tengo proyectos creados con versiones viejas del plugin. ¿Hay que migrar algo? No. Al abrir una sesion, el plugin completa solo lo que falte y, si la memoria esta vacia, te ofrece una vez llenarla.

¿Como me entero de que salio una version nueva? Automatico: al abrir una sesion el plugin compara tu version con la publicada (una vez cada 6 horas) y te avisa, preguntando si quieres actualizar.

Si algo no anda

Actualice el plugin pero el equipo sigue igual. Cierra y abre la sesion. Todo lo nuevo se carga al inicio.

La oficina virtual se ve vacia. Casi siempre es que no se reinicio la sesion despues de actualizar. Se llena a medida que el equipo trabaja.

Un integrante quiso pedirle ayuda a otro y fue bloqueado. Correcto, es el diseño: solo el jefe de equipo puede hacerlo. El integrante hace su trabajo directo o deja una nota al jefe.

Pruebas no quiere validar. Tambien correcto si falta el informe de conformidad o el sistema no esta completo en la computadora del desarrollador (regla de oro). Pide a quien instalo que emita el informe.

Un pase fue rechazado. Es el control funcionando. El rechazo indica archivo, linea y regla; lo corrige el encargado de datos y se vuelve a revisar.

Los integrantes tardan en arrancar. Abre la sesion desde la carpeta que contiene todo el proyecto y manten la memoria al dia (`/dev-team:wiki ingest`). Con eso el arranque lee una pagina en vez de muchos archivos.

Para problemas de instalacion o configuracion, ver el [Anexo E](#).

Anexo tecnico (para quien instala y configura)

A. Instalacion y actualizacion

```
# 1. Agregar el marketplace (una sola vez)
/plugin marketplace add faast-app/faast-claude-marketplace

# 2. Instalar el plugin
/plugin install dev-team@faast-marketplace

# 3. Actualizar cuando haya cambios publicados
claude plugin marketplace update faast-marketplace
claude plugin update dev-team@faast-marketplace
# → reiniciar la sesion de Claude Code despues de actualizar
```

Requisitos (el agente `setup` los valida e instala con una confirmacion): `git` · `Docker` · `Node.js` ≥ 18 · `gh` (GitHub) o `az` (Azure DevOps) · cliente de BD (`mysql/sqlcmd/psql`) · `Playwright` (el navegador de pruebas; el servidor MCP viene incluido en el plugin) · `pipx` para `Schemathesis` (contratos de API).

Si ya tenias un servidor MCP `playwright` registrado a mano en tu configuracion personal, quitalo para no duplicar herramientas: `claude mcp remove playwright`. El del plugin queda.

B. Configuracion del proyecto

Todo vive en `.coordination/config.json` (lo crean `new-project` y `onboard`):

```

{
  "project": "backoffice",
  "topology": "multi",
  "tracker": {
    "provider": "azure",
    "azure": { "org": "{org}", "project": "{proyecto}" },
    "reviewer": "{email-del-reviewer}",
    "overheadEpicId": 1234,
    "areaPath": "{Proyecto}\\{Equipo}",
    "iterationPath": "{Proyecto}\\Sprint {N}"
  },
  "git": {
    "defaultBranch": "develop",
    "identity": { "name": "{Nombre Apellido}", "email": "{email-del-proyecto}" }
  },
  "pase": {
    "templatePath": "(opcional – default: plantilla del plugin)",
    "outputDir": "(opcional – default: .coordination/pases/)",
    "elaboradoPor": "{Nombre Apellido}",
    "to": ["mesa-de-servicio@...", "plataformas@..."],
    "cc": ["dev@...", "qa@..."],
    "aprobador": "{Nombre del que da el V.B}"
  },
  "team": { "models": { "architect": "opus" } },
  "urls": { "dev": "http://localhost:3000" },
  "e2e": { "exists": true, "path": "e2e" }
}

```

Campo	Efecto
topology	mono (un repo, servicios como carpetas) / multi (carpeta paraguas con un repo por servicio). Define como trabajan los agentes y donde vive .coordination/
tracker.provider	github / azure : donde viven HUs, items y PRs
tracker.reviewer	Reviewer que los devs ponen en cada PR
tracker.overheadEpicId	Epica/PBI padre para fixes sueltos sin epica propia
git.defaultBranch	Rama base obligatoria para ramificar (via git fetch origin)
git.identity	Identidad de commits del proyecto (nunca la default del agente)
pase.*	Plantilla, carpeta de salida, "elaborado por", destinatarios (to / cc) y aprobador del correo de pase
team.models.{agente}	Override de modelo en este proyecto (no aplica a setup/tech-writer). Personal: ~/.claude/dev-team.config.json ; proyecto gana
urls.dev	URL del ambiente que QA usa para validar
e2e.path	Donde vive la suite de pruebas (mono: e2e/ ; multi: repo {proyecto}-e2e), creada desde templates/e2e-faast/

Secretos que nunca entran a git (los templates ya los ignoran): `.coordination/dba-access.json`, `.coordination/qa-secrets.env`, `.coordination/setup-status.json`, y toda la carpeta `.coordination/evidence/`.

C. La carpeta de coordinacion

`.coordination/` es la memoria compartida del equipo. Vive en la raiz del mono-repo o en la carpeta paraguas del multi-repo. La **canonica** es la que tiene `config.json`; los repos fuera del paraguas apuntan a ella con un archivo `.coordination-root` (una linea con la ruta; gitignored).

```
.coordination/
├─ config.json           fuente de verdad (arriba)
├─ handoffs/ (+archive/) comunicacion entre agentes ({de}-to-{para}-{fecha}.md)
├─ wiki/                 memoria del proyecto (vault de Obsidian; solo escribe tech-
writer)
├─ metrics/             activity.jsonl (eventos via hooks + verdict/reopened
manuales)
├─ evidence/            evidencia QA por HU/BUG (informe-qa.md, capturas, trace,
clips) – gitignored
├─ pases/               carpetas de pase entregadas
├─ office/              la oficina virtual (se instala con /team-office)
├─ test-plans/          planes de prueba de QA
├─ design/              propuestas del ui-designer
├─ backlog.md · sprint-actual.md · architecture.md · repos.md
├─ dba-access.json      credenciales BD – NUNCA en git
├─ qa-secrets.env       credenciales de QA – NUNCA en git
└─ setup-status.json    estado del entorno – NUNCA en git
```

Evidencia en GitHub: rama huérfana `evidence` (permanente, jamás mergeada), trabajada desde un worktree aparte, estructura `evidence/issues/<n>-<slug>/` y `evidence/smokes/<fecha>-<nombre>/`, embed ``. **En Azure DevOps:** attachment via API + `<img>` en el HTML del WI. Un PR que incluya `.coordination/evidence/` o imágenes de evidencia se bloquea en `/review-pr`.

D. Cuidar el consumo

Revisa tu panel con `/usage`. Lo que más ahorra:

1. **Sesiones cortas por tarea.** Una sesión de 3 horas lee 150k+ tokens en cada turno. `/compact` a mitad de una tarea larga; `/clear` o sesión nueva al cambiar de tema.
2. **Wiki al día** (`/dev-team:wiki ingest` al cierre): la sesión siguiente arranca leyendo una página de 2-3k tokens en vez del historial.
3. **Pregunta directa = respuesta directa**, sin `/start` para preguntas simples.
4. **Modelo principal en sonnet** (`/model sonnet`).
5. **Solo los MCP necesarios** (`claude mcp list`).

Protecciones automaticas del plugin: los subagentes no pueden delegar (solo el lead; un hook `PreToolUse` lo bloquea), los comandos corren inline (nunca como subagentes), el logging de actividad va por hooks (cero tokens) y los agentes usan contexto bajo demanda (si el handoff trae todo, trabajan sin releer).

Leyendo /usage : "subagents under dev-team:backend" alto = version vieja del plugin (actualizar y reiniciar); "% at >150k context" alto = sesiones demasiado largas; la primera interaccion tras el reset siempre pesa mas (cache fria).

E. Solucion de problemas tecnicos

"Failed to update plugin: Plugin dev-team not found" → nombre completo: `claude plugin update dev-team@faast-marketplace`.

Actualice pero los agentes siguen igual → agentes, hooks y MCP del plugin cargan al inicio de sesion. Reiniciar. Verificar con `claude plugin list`.

QA no ve las tools `browser_*` / aparecen dos servidores playwright → desde v2.8.0 el MCP viene en el plugin (`plugins/dev-team/.mcp.json`: `@playwright/mcp` fijado, `--caps=testing,devtools,vision`, `--isolated`, salida en `.coordination/evidence/_mcp/`). Si `claude mcp list` muestra dos playwright, el personal se quita con `claude mcp remove playwright`. Si no muestra ninguno, la sesion es vieja: reiniciar. `/dev-team:setup playwright` hace este diagnostico.

La oficina virtual se ve vacia → (1) ¿sesion reiniciada tras actualizar? (2) ¿el server apunta al `.coordination` correcto? (`node .coordination/office/server.mjs --dir .coordination`) (3) `python3 --version`: los hooks lo necesitan.

Handoffs o metricas aparecen en una `.coordination` equivocada (dentro de un repo) → hay una `.coordination` huérfana (sin `config.json`) mas cerca del cwd que la canonica. Desde v2.6.11 el plugin resuelve por canonicidad y, al abrir sesion sobre una huérfana, propone fusionarla, eliminarla y crear el puntero `.coordination-root`. A mano: crear `.coordination-root` en la raiz del repo con la ruta a la carpeta paraguas, fusionar lo util y borrar el desvio.

El architect no uso opus aunque lo configure → el override lo aplica quien invoca (lead/flujo) leyendo el config: verificar la clave exacta `team.models.architect` en `.coordination/config.json` (proyecto) o `~/claude/dev-team.config.json` (personal), y que la sesion sea nueva.

Los agentes tardan en arrancar → (1) sesion abierta desde la carpeta paraguas + `/add-dir` para los repos; (2) `repos.md` con las rutas locales reales; (3) wiki al dia.

Un PR fue bloqueado por "evidencia en el PR" → el diff incluye `.coordination/evidence/` o imagenes/clips de QA. Retirarlos del PR (la evidencia va solo a la rama `evidence` o al tracker) y verificar que `.coordination/evidence/` este en el `.gitignore` del repo.

Un pase fue rechazado por el release-manager → el handoff de rechazo lista archivo+linea+regla; lo corrige el DBA y se re-audita. Nunca se "arregla" editando scripts ya aplicados.